

Sprint 4 Structured Notes: Numbers, Booleans, Operators, and Control Flow

0. What Sprint 4 Is About

Sprint 4 teaches JavaScript how to **calculate**, **compare**, and **decide**.

In earlier sprints, you learned that JavaScript works with values, variables, types, and strings. Sprint 4 uses those ideas to make programs behave differently depending on data.

Main goal:

Use math, comparisons, conditions, and logical decisions correctly.

By the end of Sprint 4, you should understand:

- the `Number` type
 - integers, decimals, negative numbers, `Infinity`, and `NaN`
 - arithmetic operators
 - type coercion with numbers, strings, and booleans
 - operator precedence
 - increment and decrement
 - compound assignment
 - booleans
 - equality and inequality
 - comparison operators
 - unary operators
 - bitwise operators at a basic awareness level
 - `if`, `else if`, and `else`
 - logical operators
 - truthy and falsy values
 - the `Math` object
 - `isNaN()`
 - `parseInt()` and `parseFloat()`
 - `toFixed()`
 - `null` and `undefined` comparisons
 - `switch`
 - beginner calculator logic
-

Part A: The Core Idea

1. Calculate, Compare, Decide

Sprint 4 has three main actions.

Calculate

```
const price = 10;
const quantity = 3;
const total = price * quantity;

console.log(total); // 30
```

Compare

```
const score = 85;
const isPassing = score >= 70;

console.log(isPassing); // true
```

Decide

```
const score = 85;

if (score >= 70) {
  console.log("You passed");
} else {
  console.log("Keep practicing");
}
```

Expected output:

```
You passed
```

Simple meaning:

JavaScript can use values to choose what code should run.

Part B: Numbers

2. The Number Type

JavaScript uses the `Number` type for normal numeric values.

```
const age = 16;  
const price = 19.99;  
const temperature = -5;  
const zero = 0;  
  
console.log(typeof age); // number
```

Numbers are used for:

- age
- score
- price
- quantity
- total
- temperature
- count
- page number

3. Integers

An integer is a whole number.

```
const students = 25;  
const score = 90;  
const page = 1;
```

4. Decimals

A decimal number has a decimal point.

```
const price = 19.99;  
const taxRate = 0.08;  
const average = 87.5;
```

Decimals are still type `number`.

```
console.log(typeof 19.99); // number
```

5. Negative Numbers

JavaScript can use negative numbers.

```
const temperature = -5;  
const balance = -20;
```

6. Infinity

`Infinity` is a special numeric value.

```
console.log(10 / 0); // Infinity  
console.log(typeof Infinity); // number
```

Simple meaning:

JavaScript represents some impossible or extremely large numeric results as `Infinity`.

7. NaN

`NaN` means **Not a Number**.

```
console.log("hello" * 2); // NaN  
console.log(typeof NaN); // number
```

This is confusing but important:

`NaN` means the numeric calculation failed, but its JavaScript type is still `number`.

Beginner rule:

If math gives you `NaN`, check whether one of your values is not valid numeric data.

8. Different Number Notations

JavaScript supports different ways to write numbers.

Decimal

Normal base-10 numbers:

```
const decimal = 42;
```

Binary

Binary uses base 2. It starts with `0b`.

```
const binary = 0b1010;  
console.log(binary); // 10
```

Octal

Octal uses base 8. It starts with `0o`.

```
const octal = 0o12;  
console.log(octal); // 10
```

Hexadecimal

Hexadecimal uses base 16. It starts with `0x`.

```
const hex = 0xA;  
console.log(hex); // 10
```

Beginner note:

You will mostly use normal decimal numbers, but you should recognize the other forms.

Part C: Arithmetic Operators

9. Basic Arithmetic Operators

Arithmetic operators do math.

```
console.log(10 + 3); // 13 addition  
console.log(10 - 3); // 7 subtraction
```

```
console.log(10 * 3); // 30 multiplication
console.log(10 / 3); // 3.3333333333333335 division
console.log(10 % 3); // 1 remainder
console.log(2 ** 3); // 8 exponentiation
```

10. Addition: `+`

```
console.log(5 + 10); // 15
```

With two numbers, `+` means addition.

11. Subtraction: `-`

```
console.log(10 - 5); // 5
```

12. Multiplication: `*`

```
console.log(4 * 3); // 12
```

13. Division: `/`

```
console.log(10 / 2); // 5
```

14. Remainder: `%`

The `%` operator gives the remainder after division.

```
console.log(10 % 3); // 1
```

Because:

```
10 divided by 3 = 3 remainder 1
```

Common use: checking even or odd numbers.

```
const number = 8;
```

```
if (number % 2 === 0) {  
  console.log("Even");  
} else {  
  console.log("Odd");  
}
```

Expected output:

Even

15. Exponentiation: `**`

The `**` operator raises a number to a power.

```
console.log(2 ** 3); // 8  
console.log(5 ** 2); // 25
```

Simple meaning:

$2 ** 3 = 2 * 2 * 2 = 8$

Part D: Operator Precedence

16. What Is Operator Precedence?

Operator precedence means JavaScript follows an order of operations.

```
const total = 10 + 5 * 2;  
console.log(total); // 20
```

Why not `30`?

Because multiplication happens before addition.

$5 * 2 = 10$
 $10 + 10 = 20$

17. Parentheses

Parentheses make your intention clear.

```
const total = (10 + 5) * 2;  
console.log(total); // 30
```

Now JavaScript does the parentheses first.

Beginner rule:

Use parentheses when the expression could be misunderstood.

Part E: Type Coercion

18. What Is Type Coercion?

Type coercion means JavaScript automatically converts one type into another.

This can be useful, but it can also create bugs.

19. **+** with Numbers and Strings

```
console.log(5 + "10");
```

Expected output:

```
510
```

Why?

Because one value is a string. With **+**, JavaScript often changes the number into text and joins the values.

```
5 + "10"  
"5" + "10"  
"510"
```


20. -, *, and / with Numeric Strings

```
console.log("10" - 5); // 5
console.log("10" * 2); // 20
console.log("10" / 2); // 5
```

JavaScript tries to convert the string `"10"` into the number `10`.

Beginner rule:

Do not rely too much on automatic conversion. Convert values yourself when you want numbers.

Better:

```
const input = "10";
const number = Number(input);

console.log(number + 5); // 15
```

21. Booleans in Arithmetic

Booleans can be coerced into numbers.

```
console.log(true + 2); // 3
console.log(false * 10); // 0
```

Why?

```
true becomes 1
false becomes 0
```

This is legal JavaScript, but beginner code is usually clearer when you avoid depending on it.

Part F: Increment, Decrement, and Compound Assignment

22. Increment: ++

Increment means increase by 1.

```
let count = 0;
count++;

console.log(count); // 1
```

This is similar to:

```
count = count + 1;
```

23. Decrement: --

Decrement means decrease by 1.

```
let count = 3;
count--;

console.log(count); // 2
```

This is similar to:

```
count = count - 1;
```

24. Prefix and Postfix Basic Awareness

You may see ++count and count++.

```
let count = 1;
console.log(count++); // 1
console.log(count);   // 2
```

Postfix returns the old value first, then updates.

```
let count = 1;
console.log(++count); // 2
console.log(count);   // 2
```

Prefix updates first, then returns the new value.

Beginner rule:

Use increment on its own line to avoid confusion.

Clear version:

```
count++;
console.log(count);
```

25. Compound Assignment Operators

Compound assignment updates a variable using an operation.

```
let score = 10;
score += 5;
console.log(score); // 15
```

This means:

```
score = score + 5;
```

Common compound assignments:

```
let value = 10;

value += 2; // value = value + 2
value -= 2; // value = value - 2
value *= 2; // value = value * 2
value /= 2; // value = value / 2
value %= 2; // value = value % 2
value **= 2; // value = value ** 2
```

Example:

```
let total = 100;  
total -= 25;  
  
console.log(total); // 75
```

Part G: Booleans

26. What Is a Boolean?

A boolean is either:

```
true  
false
```

Booleans usually answer yes/no questions.

```
const isLoggedIn = true;  
const hasPermission = false;  
const isPassing = 85 >= 70;  
  
console.log(isPassing); // true
```

Good boolean variable names often start with:

- is
- has
- can
- should

Examples:

```
const isActive = true;  
const hasAccount = false;  
const canEdit = true;  
const shouldSave = false;
```

Part H: Equality and Inequality

27. Loose Equality: `==`

Loose equality allows type coercion.

```
console.log(5 == "5"); // true
```

Why?

JavaScript converts before comparing.

28. Strict Equality: `===`

Strict equality checks both value and type.

```
console.log(5 === "5"); // false
```

Why?

```
5 is a number  
"5" is a string
```

They look similar, but they are not the same type.

Beginner rule:

Use `===` almost always.

29. Loose Inequality: `!=`

```
console.log(5 != "5"); // false
```

This is false because loose equality says they are equal after coercion.

30. Strict Inequality: `!==`

```
console.log(5 !== "5"); // true
```

This is true because the types are different.

Beginner rule:

Prefer `!==` over `!=`.

Part I: Comparison Operators

31. Greater Than and Less Than

```
console.log(10 > 5); // true
console.log(10 < 5); // false
```

32. Greater Than or Equal To

```
console.log(10 >= 10); // true
console.log(9 >= 10); // false
```

33. Less Than or Equal To

```
console.log(5 <= 5); // true
console.log(6 <= 5); // false
```

Comparison operators return booleans.

```
const age = 18;
const canVote = age >= 18;

console.log(canVote); // true
```

Part J: Logical Operators

34. AND: `&&`

`&&` means both conditions must be true.

```
const age = 20;
const hasId = true;

if (age >= 18 && hasId) {
  console.log("Entry allowed");
}
```

Simple meaning:

age is at least 18 AND has ID

35. OR: ||

|| means at least one condition must be true.

```
const isAdmin = false;
const isOwner = true;

if (isAdmin || isOwner) {
  console.log("Can edit");
}
```

Simple meaning:

admin OR owner

36. NOT: !

! flips a boolean.

```
const isLoggedIn = false;

console.log(!isLoggedIn); // true
```

Common use:

```
if (!isLoggedIn) {  
  console.log("Please log in");  
}
```

Part K: Unary Operators

37. What Is a Unary Operator?

A unary operator works on one value.

Examples:

```
let number = 5;  
  
console.log(-number); // -5  
console.log(+ "10");  // 10  
console.log(!true);   // false
```

Common unary operators:

- `-` makes a number negative
- `+` tries to convert a value to a number
- `!` converts to boolean and flips it
- `typeof` checks a value's type

Example with `typeof`:

```
console.log(typeof 10); // number  
console.log(typeof "10"); // string
```

Part L: Bitwise Operators

38. Basic Awareness of Bitwise Operators

Bitwise operators work with the binary representation of numbers.

Examples of bitwise operators:


```
&    // AND
|    // OR
^    // XOR
~    // NOT
<<  // left shift
>>  // right shift
```

Example:

```
console.log(5 & 1); // 1
```

Beginner meaning:

Bitwise operators are advanced compared with normal arithmetic and logical operators. At this stage, you only need basic awareness that they exist and work at the binary level.

Do not confuse:

```
&& // logical AND
&  // bitwise AND
```

For beginner condition checks, use `&&`, not `&`.

Part M: Truthy and Falsy Values

39. What Are Truthy and Falsy Values?

JavaScript can treat non-boolean values like booleans in conditions.

Falsy values include:

```
false
0
""
null
undefined
NaN
```

Everything else is generally truthy.

40. Empty String Example

```
const username = "";

if (username) {
  console.log("Username exists");
} else {
  console.log("Username is missing");
}
```

Expected output:

```
Username is missing
```

Because `""` is falsy.

41. The `0` Problem

`0` is falsy, but it can be a valid value.

```
const score = 0;

if (score) {
  console.log("Score exists");
} else {
  console.log("This runs, but score is actually 0");
}
```

Expected output:

```
This runs, but score is actually 0
```

Safer check:

```
if (score !== null && score !== undefined) {
  console.log("Score exists");
}
```

Beginner rule:

Be careful when using truthy/falsy checks with values where `0`, `""`, or `false` might be valid.

Part N: Conditional Statements

42. `if`

Use `if` when code should run only when a condition is true.

```
const age = 18;

if (age >= 18) {
  console.log("Adult");
}
```

Expected output:

```
Adult
```

43. `else`

Use `else` for the alternative path.

```
const age = 16;

if (age >= 18) {
  console.log("Adult");
} else {
  console.log("Minor");
}
```

Expected output:

```
Minor
```

44. `else if`

Use `else if` when there are multiple conditions.

```
const score = 87;

if (score >= 90) {
  console.log("A");
} else if (score >= 80) {
  console.log("B");
} else if (score >= 70) {
  console.log("C");
} else {
  console.log("Keep practicing");
}
```

Expected output:

B

Why?

```
87 >= 90 is false
87 >= 80 is true
```

Once JavaScript finds a true branch, it runs that branch and skips the rest.

45. Order Matters in Conditions

This version is wrong:

```
const score = 95;

if (score >= 70) {
  console.log("C");
} else if (score >= 80) {
  console.log("B");
} else if (score >= 90) {
  console.log("A");
}
```

Expected output:

C

Why?

The first condition is already true, so JavaScript never reaches the later checks.

Correct order:

```
if (score >= 90) {  
  console.log("A");  
} else if (score >= 80) {  
  console.log("B");  
} else if (score >= 70) {  
  console.log("C");  
}
```

Beginner rule:

When checking ranges, put the most specific or highest condition first.

Part O: Switch Statements

46. What Is `switch`?

A `switch` statement compares one expression against many exact cases.

```
const day = "Monday";  
  
switch (day) {  
  case "Monday":  
    console.log("Start of the week");  
    break;  
  
  case "Friday":  
    console.log("Almost weekend");  
    break;  
  
  case "Saturday":  
  case "Sunday":  
    console.log("Weekend");  
    break;  
  
  default:
```

```
    console.log("Regular day");  
}
```

Expected output:

Start of the week

47. Why `break` Matters

`break` stops the switch after a matching case runs.

Without `break`, JavaScript can continue into the next case. This is called fall-through.

Beginner rule:

Use `break` unless you intentionally want multiple cases to run together.

48. `default`

`default` runs when no case matches.

```
const role = "guest";  
  
switch (role) {  
  case "admin":  
    console.log("Full access");  
    break;  
  
  case "editor":  
    console.log("Edit access");  
    break;  
  
  default:  
    console.log("Limited access");  
}
```

Expected output:

Limited access

49. `if/else` vs `switch`

Use `if/else` for ranges:

```
if (score >= 90) {  
  console.log("A");  
}
```

Use `switch` for exact named values:

```
switch (role) {  
  case "admin":  
  case "editor":  
  case "viewer":  
}
```

Part P: The Math Object

50. What Is `Math`?

`Math` is a built-in JavaScript object with useful math tools.

You do not create it yourself. It already exists.

51. Common `Math` Methods

`Math.round()`

Rounds to the nearest integer.

```
console.log(Math.round(4.6)); // 5  
console.log(Math.round(4.4)); // 4
```

`Math.floor()`

Rounds down.

```
console.log(Math.floor(4.9)); // 4
```

`Math.ceil()`

Rounds up.

```
console.log(Math.ceil(4.1)); // 5
```

`Math.max()`

Finds the largest number.

```
console.log(Math.max(10, 4, 25)); // 25
```

`Math.min()`

Finds the smallest number.

```
console.log(Math.min(10, 4, 25)); // 4
```

`Math.random()`

Returns a random decimal from `0` up to, but not including, `1`.

```
console.log(Math.random());
```

Random integer from 1 to 10

```
const randomNumber = Math.floor(Math.random() * 10) + 1;  
console.log(randomNumber);
```

Simple meaning:

```
Math.random() gives 0 to less than 1  
* 10 gives 0 to less than 10  
Math.floor() makes it 0 to 9  
+ 1 makes it 1 to 10
```


Part Q: Numeric Parsing and Formatting

52. User Input Often Starts as Text

Many browser inputs give you strings, even when the user types numbers.

```
const input = "42";  
console.log(typeof input); // string
```

If you want math, convert the string into a number.

53. `parseInt()`

`parseInt()` reads a whole number from a string.

```
console.log(parseInt("42")); // 42  
console.log(parseInt("42.9")); // 42
```

It stops at the decimal part.

54. `parseFloat()`

`parseFloat()` reads a decimal number from a string.

```
console.log(parseFloat("42.9")); // 42.9
```

Use `parseFloat()` when decimals matter.

55. `Number()`

`Number()` converts the whole value into a number.

```
console.log(Number("42")); // 42  
console.log(Number("42.9")); // 42.9
```

If the whole string is not numeric, the result can be `NaN`.

```
console.log(Number("hello")); // NaN
```

56. isNaN()

isNaN() checks whether a value is NaN after numeric conversion.

```
console.log(isNaN("hello")); // true
console.log(isNaN("10"));    // false
console.log(isNaN(10));      // false
```

Simple meaning:

isNaN() asks: would this fail as a number?

Beginner note:

Number.isNaN() is stricter, but for Sprint 4, understand the basic isNaN() behavior first.

57. toFixed()

toFixed() formats a number to a fixed number of decimal places.

```
const price = 10 / 3;

console.log(price);           // 3.3333333333333335
console.log(price.toFixed(2)); // "3.33"
```

Important:

toFixed() returns a string.

```
const result = price.toFixed(2);
console.log(typeof result); // string
```

Use it for display formatting, not for continued numeric calculation unless you convert it back.

Part R: null and undefined Comparisons

58. Reminder: null vs undefined

From Sprint 2:

undefined = no value assigned yet
null = intentionally empty

59. Loose Equality with `null` and `undefined`

```
console.log(null == undefined); // true
```

Loose equality treats them as equal.

60. Strict Equality with `null` and `undefined`

```
console.log(null === undefined); // false
```

Strict equality checks type, so they are not the same.

Beginner rule:

Use strict equality when the difference matters.

61. Safer Missing Value Check

Sometimes you want to check for both `null` and `undefined`.

Clear beginner version:

```
if (value === null || value === undefined) {  
  console.log("Value is missing");  
}
```

Shorter version sometimes used:

```
if (value == null) {  
  console.log("Value is null or undefined");  
}
```

Beginner advice:

Use the clear version first until you are comfortable.

Part S: Core Sprint 4 Code Pattern

62. Core Code Pattern

Type this code yourself and predict the output before running it.

```
const value = "10";

console.log(5 + value);
console.log(value - 5);

const grade = 82;

if (grade >= 90) {
  console.log("A");
} else if (grade >= 80) {
  console.log("B");
} else {
  console.log("Keep practicing");
}
```

Expected output:

```
510
5
B
```

63. Core Code Pattern Explanation

```
console.log(5 + value);
```

`value` is the string `"10"`.

With `+`, JavaScript joins text if one side is a string.

So:

```
5 + "10" = "510"
```

Next:

```
console.log(value - 5);
```

With `-`, JavaScript tries to convert `"10"` into the number `10`.

So:

```
10 - 5 = 5
```

Then:

```
const grade = 82;
```

The first condition:

```
grade >= 90
```

is false.

The second condition:

```
grade >= 80
```

is true.

So JavaScript prints:

```
B
```

Part T: Mini-Build 1: Grade Checker

64. Task

Write code that prints a letter grade based on a score.

```
const score = 87;
```

```
if (score >= 90) {
  console.log("A");
} else if (score >= 80) {
  console.log("B");
} else if (score >= 70) {
  console.log("C");
} else if (score >= 60) {
  console.log("D");
} else {
  console.log("F");
}
```

Expected output:

B

Concepts used:

- numbers
- comparison operators
- booleans
- if
- else if
- else
- condition order

Part U: Mini-Build 2: Calculator Function

65. Task

Create a calculator that accepts two numbers and an operator string.

```
function calculate(a, operator, b) {
  if (operator === "+") {
    return a + b;
  } else if (operator === "-") {
    return a - b;
  } else if (operator === "*") {
    return a * b;
  } else if (operator === "/") {
    if (b === 0) {
      return "Cannot divide by zero";
    }
  }
}
```

```

    }

    return a / b;
  } else {
    return "Unknown operator";
  }
}

console.log(calculate(10, "+", 5));
console.log(calculate(10, "-", 5));
console.log(calculate(10, "*", 5));
console.log(calculate(10, "/", 5));
console.log(calculate(10, "/", 0));

```

Expected output:

```

15
5
50
2
Cannot divide by zero

```

Concepts used:

- numbers
- arithmetic operators
- strict equality
- `if`, `else if`, `else`
- nested condition
- division-by-zero protection
- return values

Part V: Practice Set

66. Predict the Output

Before running this code, predict each result.

```

console.log(5 + "10");
console.log("10" - 5);
console.log(true + 2);
console.log(false * 10);

```

```
console.log(10 + 5 * 2);
console.log((10 + 5) * 2);
console.log(5 === "5");
console.log(5 == "5");
```

Expected output:

```
510
5
3
0
20
30
false
true
```

67. Arithmetic Drill

```
console.log(20 + 5);
console.log(20 - 5);
console.log(20 * 5);
console.log(20 / 5);
console.log(20 % 6);
console.log(2 ** 4);
```

Expected output:

```
25
15
100
4
2
16
```

68. Condition Drill

```
const age = 17;
const hasPermission = true;

if (age >= 18 || hasPermission) {
  console.log("Allowed");
} else {
```



```
    console.log("Not allowed");  
}
```

Expected output:

Allowed

Why?

`age >= 18` is false, but `hasPermission` is true. With `||`, only one side needs to be true.

Part W: Common Sprint 4 Mistakes

69. Mistake 1: Using `=` Instead of `===`

Wrong:

```
if (score = 100) {  
    console.log("Perfect");  
}
```

Problem:

`=` assigns a value. It does not compare.

Correct:

```
if (score === 100) {  
    console.log("Perfect");  
}
```

70. Mistake 2: Forgetting That Input Is Often a String

Wrong:

```
const input = "10";  
console.log(input + 5); // "105"
```

Correct:

```
const input = "10";
const number = Number(input);
console.log(number + 5); // 15
```

71. Mistake 3: Using `==` Casually

Avoid:

```
console.log(0 == false); // true
```

Prefer:

```
console.log(0 === false); // false
```

72. Mistake 4: Trusting Truthy/Falsy Too Much

Problem:

```
const score = 0;

if (score) {
  console.log("Score exists");
} else {
  console.log("Missing");
}
```

Expected output:

```
Missing
```

But `0` may be a real score.

Safer:

```
if (score !== null && score !== undefined) {
  console.log("Score exists");
}
```

73. Mistake 5: Wrong Condition Order

Wrong:

```
const score = 95;

if (score >= 70) {
  console.log("C");
} else if (score >= 90) {
  console.log("A");
}
```

Expected output:

C

Correct:

```
if (score >= 90) {
  console.log("A");
} else if (score >= 70) {
  console.log("C");
}
```

74. Mistake 6: Forgetting `break` in `switch`

Potentially wrong:

```
const role = "admin";

switch (role) {
  case "admin":
    console.log("Admin access");
  case "user":
    console.log("User access");
}
```

This may print both messages because there is no `break`.

Correct:

```
switch (role) {  
  case "admin":  
    console.log("Admin access");  
    break;  
  
  case "user":  
    console.log("User access");  
    break;  
}
```

Part X: Mental Models

75. Number Mental Model

A number is data JavaScript can calculate with.

```
10 + 5 // 15
```

But numeric-looking text is still text.

```
"10" + 5 // "105"
```

76. Boolean Mental Model

A boolean is a yes/no answer.

```
score >= 70 // true or false
```

77. Condition Mental Model

An `if` statement is a gate.

If the condition is true, enter this block.
If not, skip it or go to another block.

78. Strict Equality Mental Model

`===` asks two questions:

Are the values the same?
Are the types the same?

Both must be yes.

79. Logical Operator Mental Model

`&&` means:

Both must be true.

`||` means:

At least one must be true.

`!` means:

Flip the answer.

Part Y: Key Vocabulary

80. Vocabulary

Number

A numeric value in JavaScript.

25
3.14
-10

NaN

A special number value meaning Not a Number.

```
"hello" * 2 // NaN
```

Infinity

A special numeric value for infinity.

```
10 / 0 // Infinity
```

Arithmetic Operator

An operator that performs math.

```
+ - * / % **
```

Operator Precedence

The order in which JavaScript evaluates operators.

Type Coercion

Automatic type conversion.

```
5 + "10" // "510"
```

Boolean

A true or false value.

```
true  
false
```

Comparison Operator

An operator that compares values and returns a boolean.

```
> < >= <=
```

Strict Equality

Checks value and type.

```
===
```

Logical Operator

Combines or changes boolean logic.

```
&& || !
```

Truthy

A value that behaves like `true` in a condition.

Falsy

A value that behaves like `false` in a condition.

Condition

An expression used to decide whether code should run.

Switch

A control flow statement used to compare one value against many exact cases.

Math Object

A built-in object with useful math methods.

```
Math.round()  
Math.floor()  
Math.random()
```

parseInt()

Converts a string into a whole number.

parseFloat()

Converts a string into a decimal number.

toFixed()

Formats a number to a fixed number of decimal places and returns a string.

Part Z: Revision Questions

81. Questions

Try to answer these without looking at the notes.

1. What is the main goal of Sprint 4?
2. What is the JavaScript type of `25`?
3. What is the JavaScript type of `NaN`?
4. What does `10 % 3` return?
5. What does `2 ** 3` return?
6. Why does `10 + 5 * 2` return `20`?
7. What does type coercion mean?
8. Why does `5 + "10"` return `"510"`?
9. Why does `"10" - 5` return `5`?
10. What does `true + 2` return?
11. What is a boolean?
12. What is the difference between `==` and `===`?
13. Which equality operator should beginners usually prefer?
14. What does `!==` mean?
15. What do comparison operators return?
16. What does `&&` mean?
17. What does `||` mean?
18. What does `!` do?
19. Name six falsy values.
20. Why can checking `if (score)` be wrong when `score` is `0`?
21. What does an `if` statement do?
22. When should you use `else if`?
23. Why does condition order matter?
24. When is `switch` clearer than `if/else`?
25. Why does `break` matter in a `switch` statement?
26. What does `Math.floor()` do?
27. What does `Math.random()` return?
28. What does `parseInt("42.9")` return?
29. What does `parseFloat("42.9")` return?
30. What type does `toFixed()` return?
31. What does `null == undefined` return?
32. What does `null === undefined` return?

Part AA: Self-Check Answers

82. Answers

1. Sprint 4 teaches you to use math, comparisons, conditions, and logical decisions correctly.
2. `25` is type `number`.
3. `NaN` is also type `number`.
4. `10 % 3` returns `1`.
5. `2 ** 3` returns `8`.
6. Multiplication happens before addition.
7. Type coercion means JavaScript automatically converts one type into another.
8. Because `+` with a string usually joins values as text.
9. Because `-` tries to convert `"10"` into the number `10`.
10. `true + 2` returns `3` because `true` becomes `1`.
11. A boolean is `true` or `false`.
12. `==` allows coercion. `===` checks value and type.
13. Beginners should usually prefer `===`.
14. `!==` means not equal in value or not equal in type.
15. Comparison operators return booleans.
16. `&&` means both conditions must be true.
17. `||` means at least one condition must be true.
18. `!` flips a boolean-like value.
19. Falsy values: `false`, `0`, `""`, `null`, `undefined`, `NaN`.
20. Because `0` is falsy, but it may be a valid score.
21. An `if` statement runs code only when a condition is true.
22. Use `else if` when there are multiple conditions.
23. JavaScript runs the first true branch and skips the rest.
24. Use `switch` when comparing one value against many exact cases.
25. `break` stops the switch from continuing into later cases.
26. `Math.floor()` rounds down.
27. `Math.random()` returns a random decimal from `0` up to, but not including, `1`.
28. `parseInt("42.9")` returns `42`.
29. `parseFloat("42.9")` returns `42.9`.
30. `toFixed()` returns a string.
31. `null == undefined` returns `true`.
32. `null === undefined` returns `false`.

Part AB: Final Sprint 4 Checklist

You are ready to move on from Sprint 4 when you can do these things:

- ☐ Explain what the `Number` type is.

- [] Use integers, decimals, negative numbers, `Infinity`, and `NaN`.
- [] Explain why `typeof NaN` returns `"number"`.
- [] Use `+`, `-`, `*`, `/`, `%`, and `**`.
- [] Explain the remainder operator `%`.
- [] Explain operator precedence.
- [] Use parentheses to make expressions clear.
- [] Predict common type coercion results.
- [] Explain why `5 + "10"` returns `"510"`.
- [] Explain why `"10" - 5` returns `5`.
- [] Use `++` and `--` carefully.
- [] Use compound assignment like `+=` and `-=`.
- [] Explain booleans.
- [] Use comparison operators.
- [] Explain `==` vs `===`.
- [] Prefer `===` and `!==` in beginner code.
- [] Use `&&`, `||`, and `!`.
- [] Explain truthy and falsy values.
- [] Avoid treating valid `0` values as missing data.
- [] Write `if`, `else if`, and `else` statements.
- [] Put range conditions in the correct order.
- [] Use `switch` for exact value comparisons.
- [] Use `break` correctly in `switch`.
- [] Use common `Math` methods.
- [] Convert strings to numbers with `Number()`, `parseInt()`, or `parseFloat()`.
- [] Check invalid numeric results with `isNaN()`.
- [] Format decimals with `toFixed()`.
- [] Explain why `toFixed()` returns a string.
- [] Explain loose and strict comparisons with `null` and `undefined`.
- [] Build a simple calculator function.
- [] Reject division by zero before returning a division result.

Part AC: Minimum Passing Standard

You understand Sprint 4 if you can clearly explain this code:

```
const value = "10";

console.log(5 + value); // "510"
console.log(value - 5); // 5

const score = 87;

if (score >= 90) {
  console.log("A");
}
```

```
} else if (score >= 80) {  
  console.log("B");  
} else {  
  console.log("Keep practicing");  
}
```

Expected output:

```
510  
5  
B
```

You should be able to explain:

1. `value` stores the string `"10"`.
2. `5 + value` becomes `"510"` because `+` with a string joins text.
3. `value - 5` becomes `5` because JavaScript converts `"10"` into a number for subtraction.
4. `score >= 90` is false.
5. `score >= 80` is true.
6. JavaScript prints `B` and skips the rest.

Part AD: One-Sentence Summary

Sprint 4 teaches you how to use JavaScript numbers, booleans, operators, comparisons, and conditional statements so your programs can calculate values and choose what to do next.